SAVITRIBAI PHULE PUNE UNIVERSITY

PCC-201-COM-DS

Sem:3 7022416

Seat No S500220258
Stk. No 3497260
Sub. DS
Centre 4022

Examination	SPPU Theory Exam
Day & Date	15 December 2025
Subject	DATA STRUCTUR
Paper No.	[6582]-236 Sec
Medium of Answer	ENGLISH

Seat No. : In figure & In word

S	5	0	0	2	2	0	2	5

FIVE ZERO ZERO TW
TWO ZERO TWO FIVE
EIGHT

Signature of Candidate

Instruction to Candidate

1. Candidate has to confirm seat number, subject and centre number printed on Bar code and write it on attendance sheet.
   विद्यार्थ्याने प्रथम बार कोडवरील आसन क्रमांक, विषय व केंद्र क्रमांक तपासून योग्य असल्याची खात्री करावी आणि उपस्थिती पत्रकावर नोंदवावी.
2. Paste Bar Code in prescribed space.
   उत्तरपत्रिकेवरील विहित जागेवर बार कोड लावावा.
3. Do not write anything on Bar code sticker, otherwise it will be treated as unfairmeans.
   बार कोड स्टिकरवर काहीही लिहू नये, अन्यथा परीक्षा गैरप्रकार समजला जाईल.

Q. No.	Examiner	Moderator
1	10	10
2		
3		
4	08	12
5	12	12
6		
7		
8	10	10
9		
10	12	12
11		
12		
Total in Figure	52	056
Total in Words	Fifty Two	Five six
Signature		

(in Ink)

Total	Marks in Figure	Marks in Words	Sign
Examiner	52	Fifty Two	
Moderator	56	Five six	

१. विद्यार्थ्याने उत्तरपत्रिकेच्या मुखपृष्ठावर तसेच उपस्थिती पत्रकावर विहित जागेत आसन क्रमांक अंकात व अक्षरात बिनचूक लिहून स्वाक्षरी करावी.

२. उत्तरपत्रिकेवर फक्त निळ्या अथवा काळ्या शाईचा उपयोग करावा, अन्यथा उत्तरपत्रिकेचे मूल्यमापन केले जाणार नाही.

३. उत्तरपत्रिकेच्या पृष्ठक्रमांक ३ पासून लिहिण्यास प्रारंभ करावा.

४. संबंधित प्रश्नाचे अथवा उपप्रश्नाचे उत्तर जेथून सुरू होते तेथेच समासात प्रश्न क्रमांक, उपप्रश्न क्रमांक अचूक व स्पष्ट लिहावा, यासाठी वेगळ्या शाईचा उपयोग करू नये.

५. प्रत्येक पानाच्या दोन्ही बाजूस लिहावे, उत्तरपत्रिका किंवा पुरवणी उत्तरपत्रिकेचे कोणतेही पान फाडू नये, फाडल्यास परीक्षा गैरप्रकार समजून पुढील कार्यवाही करण्यात येईल.

६. पेपर संपण्यापूर्वी १० मिनिटे अगोदर इशारा घंटा होईल, त्यानंतर विद्यार्थ्याने उत्तरपत्रिका व पुरवणी उत्तरपत्रिकेवर होलोक्राफ्ट स्टिकर विहित जागेवरच लावावा.

७. कॉपी करणे किंवा दुसऱ्याच्या नावावर परीक्षेस बसणे यांसारख्या कृती 'महाराष्ट्र-प्रीव्हेन्शन ऑफ मालप्रॅक्टिस ॲक्ट युनिव्हर्सिटी, बोर्ड ॲण्ड अदर स्पेसिफाईड एक्झामिनेशन्स ॲक्ट, १९८२' (सा.फु.पु.वि. चा अध्यादेश क्रमांक ९) त्यानुसार संमत केलेला कायदा या अन्वये दंडही असेल.

Candidate shall fill all information about se number, paper etc. in prescribed space and sign o the answer book and attendance sheet.

Candidate shall use blue or black ink onl Otherwise answer book will not be evaluated.

Candidate shall start writing answer from page n 3 of the answer book.

Candidate shall mention question number, su question number correctly at the beginning of th same and shall not use ink other than blue or blac

Candidate shall write on both sides of pages an shall not tear off any page, it will be treated a unfairmeans.

Warning bell will be given before 10 minutes of th concluding time. Candidate shall paste Holocra Sticker at appropriate space on the answer book

An Act of Copying or Impersonations at a Examination is Punishable under 'The Maharashtr Prevention of Malpractices at University, Board an Other Specified Examinations Act, 1982' (Ordinanc 9 of SPPU). The Act passed to the effect.

Examiner and Moderator has to write marks on all given appropriate place only. Examiner should give assessment tick(✓) or (×) in the margin.

Q. No.	Examiner	Moderator	Verification	Revaluation
1	10	10		
2				
3				
4	8	12		
5	12	12		
6				
7				
8	10	10		
9				
10	12	12		
11				
12				
Total	52	56		

Q.No.

Q1.

(a). Time complexity of a program or algorithm is the growth or rate of that algorithm or program as the size of input increases.

- Asymptotic notations are different ways of representing the time complexity of algorithms.

- These notations depend on different assumptions of the input and the primitive operations such as comparisons, assignments, arithmetic modifications and so on.

- Primitive operations are low-level operations that have an execution time of 1, ~~i.e.~~ that is, constant execution time.

- The three types of Asymptotic notations are:

i) Worst-case time complexity : Big-O Notation - [O]
- This notation represents the upper bound, that is, the maximum time that the program will take to execute.
- This is considered as the worst-case time complexity.

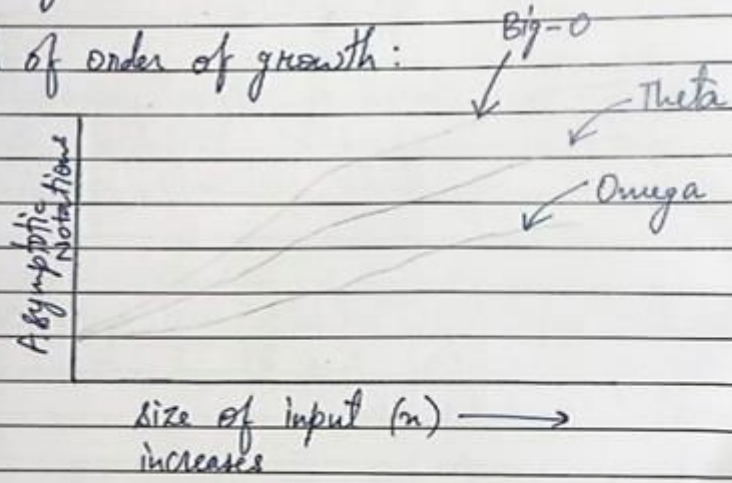
ii) Omega Notation [Ω]:
- This notation represents the lower bound, that is the minimum time of execution of the program for given input size 'n'.
- This is the best-case time complexity.

प्र. क्र./Q.No.

iii. Theta Notation [Θ]:

- This notation represents the exact bound, that is the approximate exact time of execution of the program.
- This is considered as the average case time complexity.

Graph of order of growth:



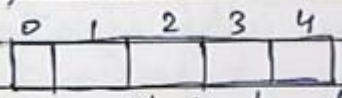
5

b) This graph shows that ~~a~~ for any given input size n:

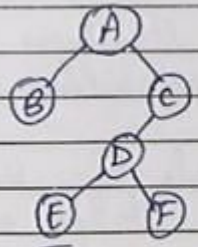
- The time ~~complexity~~ of Big-O Notation will be maximum.
- The time ~~complexity~~ of Omega Notation will be least.
- The time ~~complexity~~ of Theta Notation will be closest to the actual execution time and will be between Big-O and Omega.

Q.No. (b) - Q.81.

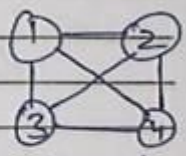
Linear Data Structures	Non-Linear Data Structures
i. These data structures store data in sequential order or linear order.	These do not store data in sequential order but in rather, in non-linear, hierarchical order.
ii. The data is stored one after the other, forming a single sequence. ~~that~~	The data is stored in non-linear format.
iii. An element can have a maximum of one successor and one predecessor.	An element can have one ~~or~~ parent and multiple children and/or siblings.
iv. Traversal of data is easy to implement.	Traversal of data is complex and requires more operations.
v. Examples: Arrays, Linked List, Stacks, Queues.	Examples: ~~Graph~~ Trees, and Graphs.



Array with 4 elements.



Tree



Graph

SPPU- 6/ 36

Q.No.
E
M

Savitribai Phule Pune Uni

प्र. क्र./Q.No.

- ADT or Abstract Data Type are types of data types that provide only the essential information and hide the other details.

- ADTs are a type of object whose logical behaviour is defined by a set of values and operations that are hidden.

- ADT focuses on ~~the~~ what to do rather than how to do it and are called abstract because they provide an implementation-independent view.

- The implementation of ADTs require special data structures to store the data and algorithms to perform basic operations.

- Data Structures are an implementation of Abstract Data Type.

- They provide ~~only the~~ ~~the~~ the 'how to do' by ~~st~~ defining how the data is stored in memory.

- Data Structures also use certain algorithms that are hidden from view, to perform operations.

- Example of such a data structure is stack which stores elements in last in, first out manner.

Q.No.

- Stack performs ADT operations lik push(), pop(), top(), isEmpty() ~~and~~, for insertion, deletion, getting top-most value ~~res~~ respectively.

Q4.

(a). ADT ~~=~~ or Abstract ~~of~~ Data Type is a data type that only gives essential ~~det~~ information and hides implementation details.

- Arrays are also an implementation of Abstract Data Type. ~~since they~~

- Array as ADT is the following:

i. Arrays store elements sequentially in contiguous memory locations, that can be ~~b~~ accessed by traversing through the array.

ii. Arrays have fixed spaces; since arrays are static data types, they allocate memory at compile-time and this size cannot be altered.

iii. Arrays also provides Random Access to elements.

- The operations performed on arrays are:

i) Insertion:

- Insertion in an array is done by traversing through the array to get to desired position and inserting the element

SPPU-8/36
Q.No.
E
M
Savitribai Phule Pune Univ

प्र. क्र./Q.No.

- This operation has a best-case time complexity of 1, that is, if element is to be inserted at the start of the array. O(1)

- The worst-case time complexity is n when the element has to be inserted at the end. O(n), when where n = number of elements in array.

ii) Search -:

- Searching in an array can be done by traversing through the arrays using the following:

a) Linear Search - sequentially searching all elements to find the target. ~~Time~~
- Time Complexity: O(n), when element is at last position.

b) Binary Search - ~~exit dividi~~ finding the middle element of dividing the elements into two halves continuously till target is found.
- Time Complexity: O(log n)

iii) Deletion:

- ~~A~~ Requires traversing the array to find element to be deleted and deleting it if found.
- Time Complexity: O(n)

Q.No.

iv) Sorting :

- ~~Use~~ Sorts the elements of the array in ascending or descending order.
- Uses multiple algorithms or ~~techn~~ techniques for sorting :

a) Selection Sort → Time complexity : $O(n^2)$

b) Bubble Sort → Time complexity : $O(n^2)$

c) Quick Sort → Time Complexity : $O(n \log n)$

Q4. (b) • Bubble Sort is a sorting technique or algorithm that sorts elements in ascending or descending order depending on the requirement.

• Bubble sort ~~com~~ compares adjacent elements and swaps them if they are not in the correct order.

• This sorting algorithm takes $(n-1)$ passes to complete where n = number of elements to be sorted.

• After each pass, one element is sorted, for example, if sorting in ascending order is required, then after the first pass, the largest element will be sorted to the end of the sequence.

• Time Complexity of Bubble Sort is $O(n^2)$ in worst-case scenario, but this can be optimised to $O(n)$ for best-case scenario.

SPPU-10/35

Q.No. E M

Savitribai Phule Pune Univ

प्र. क्र./Q.No.

- Best-case scenario is when the elements are already sorted.

list = [7, 4, 10, 6, 3, 12, 1, 8, 2, 15]

Step 1: Compare 7 & 4.
Since 7 > 4 thus, swap them.
⇒ 4, 7

- Comparing 7 and 10 → no change.
- Comparing 10 and 6 ⇒ 10 > 6, thus swap
- ~~Comp~~ ⇒ 4, 7, ~~10~~, 6, 10

- Comparing 10 and 3 ⇒ 10 > 3, thus swap.
  ⇒ 4, 7, 6, 3, 10

- Comparing 10 and 12 ⇒ no change
  ⇒ 4, 7, 6, 3, 10, 12

- Comparing 12 and 1 ⇒ 12 > 1, thus swap
  ⇒ 4, 7, 6, 3, 10, 1, 12

- Comparing 12 and 8 ⇒ 12 > 8, swap.
  ⇒ 4, 7, 6, 3, 10, 1, 8, 12

- Comparing 12 and 2 ⇒ ~~no change~~. 12 > 2, swap.
  ⇒ 4, 7, 6, 3, 10, 8, 2, 12

- Comparing 12 and 15 ⇒ no change.
  ⇒ 4, 7, 6, 3, 10, 8, 2, 12, 15

. ₹./Q.No.

- Thus, after 1st pass the array is:
  4, 7, 6, 3, 10, 8, 2, 12, 15
- In this, the largest element, 15, is placed in last position.

Step 2:

- Compare 4 and 7 ⇒ no change
- Compare 7 and 6 ⇒ swap
  ⇒ 4, 6, 7, 3, 10, 8, 2, 12, 15
- Compare 7 and 3 ⇒ ~~no change~~ swap
  ⇒ 4, 6, 3, 7, 10, 8, 2, 12, 15
- Compare 7 and 10 ⇒ no change
- Compare 10 and 8 ⇒ swap
  ⇒ 4, 6, 3, 7, 8, 10, 2, 12, 15
- Compare 10 and 2 ⇒ swap
  ⇒ 4, 6, 3, 7, 8, 2, 10, 12, 15
- Compare 10 and 12 ⇒ no change.

Thus, after 2nd pass, the second largest element is in right place.

4, 6, 3, 7, 8, 2, 10, 12, 15

SPPU- 12/ 36

	Q.No.						TOTAL
E							
M							

Savitribai Phule Pune

प्र. क्र./Q.No.

4, 6, 3, 7, 8, 2, 10, 12, 15

Step 3:

- Compare 4 and 6 ⇒ no change
- 3 and 6 ⇒ swap
  ⇒ 4, 3, 6, 7, 8, 2, 10, 12, 15
- 6 and 7 ⇒ no change
- 7 and 8 ⇒ no change
- 8 and 2 ⇒ swap
  ⇒ 4, 3, 6, 7, 2, 8, 10, 12, 15
- 8 and 10 ⇒ no change.

after 3rd pass, 3rd largest element is sorted.

Step 4:

- 4 and 3 ⇒ swap
  ⇒ 3, 4, 6, 7, 2, 8, 10, 12, 15
- 4 and 6 ⇒ no change
- 6 and 7 ⇒ no change
- 7 and 2 ⇒ swap
  ⇒ 3, 4, 6, ~~7~~, 2, 7, 8, 10, 12, 15

S./Q.No.

After 4th pass, 4th largest is in place.

Step 5:

- 3 & 4 ⇒ no change
- 4 & 6 ⇒ no change
- 6 & 2 ⇒ swap
  ⇒ 3, 4, 6, 2, 6, 7, 8, 10, 12, 15

After 5th pass, 5th largest, 7, is in place.

Step 6:

- 3 & 4 ⇒ no change
- 4 & 2 ⇒ swap
  ⇒ 3, 2, 4, 6, 7, 8, 10, 12, 15

08

- Remaining ⇒ no change

After 6th pass, 6 is in place.

Step 7:

- 3 & 2 ⇒ swap
  ⇒ 2, 3, 4, 6, 7, 8, 10, 12, 15

Remaining no change.

प्र. क्र./Q.No.

- The next two steps will not cause any changes and the array is sorted, in 7 passes, but total 9 passes will happen.

Q5: (a)

i) • SLL or Singly Linked List is a data structure that is linear and stores data in sequential order.

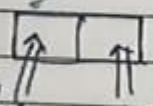
- SLL has nodes and every node has at maximum one successor. All nodes are connected to each other in memory.

- SLL may or may not have nodes stored in contiguous memory locations but still, they can be accessed since each node has the memory address pointer to its next node.

- Example:

element: (2) — (4) — (6) — (8) — X
memory location: 101 5002 31 400

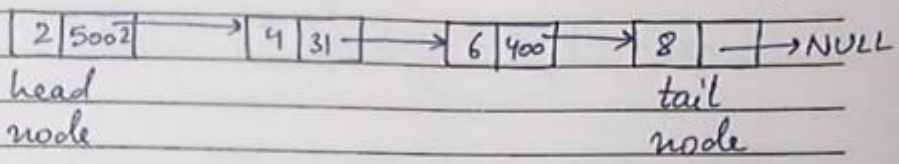
- The structure of nodes in SLL is:



Value of element — pointer which points to the memory location of next node

Q.No.

- This is what the example looks like in memory:



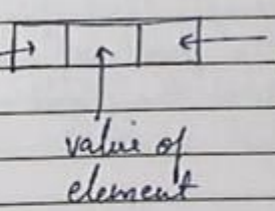
ii) • DLL or Doubly Linked List is a data structure that is similar to SLL.

- In DLL, every node is connected to its next node as well as to its previous node.

- In DLL also, a node has a maximum of one successor and one predecessor and the nodes may or may not be stored in contiguous memory locations.

- ~~Example:~~ In DLL structure, every node stores its value, the pointer to the memory address of next node as well as previous node.

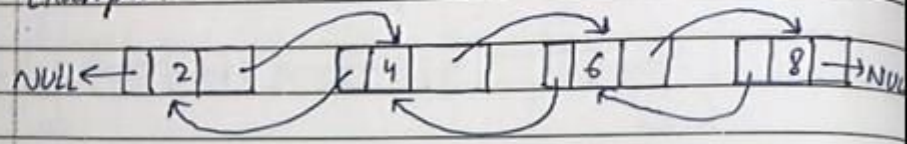
- ~~Example~~ Structure of DLL:



- So, DLL stores two pointers unlike SLL which only stores one pointer.

प्र. क्र./Q.No.

- Example:



iii). CLL or Circular linked list is a type of linked list data structure ~~that stores~~ in which the first node is connected to the last node and last node is connected to first node.

- It is more efficient since jumping from first node to last and vice-versa is a constant time operation (O(1)).

- It is of two types:

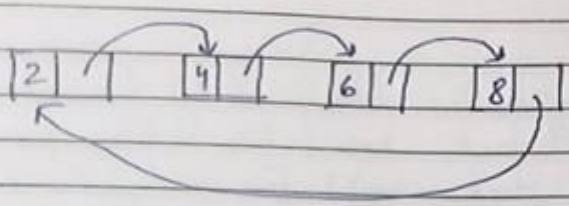
a) SCLL or Singularly Circular Linked List which is same as SLL except that first and last nodes are connected. The front of last node points to the first node.

b) DCLL or Doubly Circular Linked List which is same as DLL except that first and last nodes are connected. The front of last nodes points to first node and the ~~front of~~ back of first node points to last node.

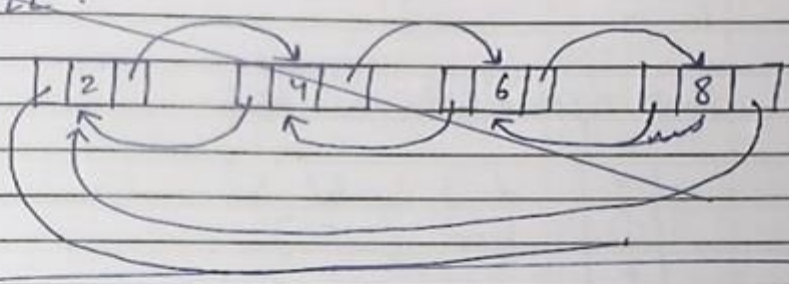
- Examples:

a) SCLL

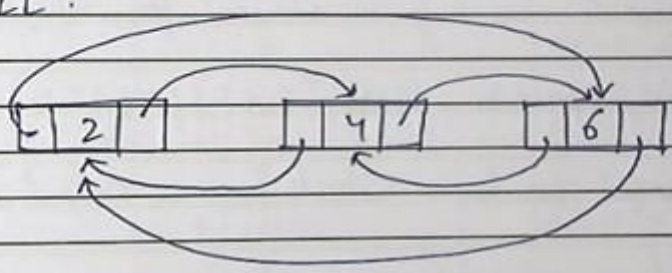
प्र. क्र./Q.No.



b) DCLL :



b) DCLL :



Q5-

(b) · Stack is an Abstract Data Type since it only gives the essential information and hides the implementation details.

· Stack as an ADT ~~is due to~~ allows for :

i) Storage of elements in a sequential manner.

ii) LIFO access - which means Last In, First Out. Due to this the element that goes in last is the first to be deleted.

प्र. क्र./Q.No.

iii. Stack operations like push() for inserting to the top of stack, pop() for deleting from the top of the stack, top() for finding the value of the element at the top (also called peek()) and so on.

Stack representation as ADT:

d
c
b
a

d is inserted last and removed first (LIFO).

It is like a stack of coins or books in which elements can only be accessed from the top.

Infix Expression: $(A+B)*D+E/(E+A*D)+C$

Scanned character	Stack	Output Answer
(	(	
~~A~~		
A	(	A
+	(+	A
B	(+	AB
)		AB+
*	*	AB+

Scanned char	Stack	Output Answer
D	*	AB+D
+	+	AB+D*
E	+	AB+D*E
/	+/	AB+D*E
(	+/(	AB+D*E
F	+/(	AB+D*EF
+	+/(+	AB+D*EF
A	+/(+	AB+D*EFA
*	+/(+*	AB+D*EFA
D	+/(+*	AB+D*EFAD
)	+/	AB+D*EFAD*+
+	+	AB+D*EFAD*+/+
C	+	AB+D*EFAD*+/+C
		AB+D*EFAD*+/+C+
		(Answer)

6

Q. K./Q.No.

Q8. (a)

- Hashing is a ~~s~~ technique which enables us to convert a large amount of data into a fixed or small value.

- It is a searching technique which allows ~~look~~ constant ~~to~~ time lookups, that is, O(1), time complexity.

- Hashing uses a hash function to input keys and perform some mathematical operation on them and give hash values which are the indices of keys into a hash table.

- The hash table stores ~~elements~~ data, in the form of key and values and is used in database indexing, caching and implementing data structures like map, set and so on.

- Hashing techniques are methods to obtain a hash value by performing mathematical operations on the keys and some of them are:

i) Division method :-

- In this method the hash value is generated by taking the modulus (remainder ~~up~~ on division) of the input key with the size of the table.

- $H(k) = k \% M$

Here, $H(k)$ is the hash function which gives a hash value.
k is the input key
M is the size of the ~~tab~~ hash table.

- It is a standard hashing technique, but may ~~to~~ lead to collision when more than one elements with the same modulo value are inserted into the same location.

ii) Folding method :

- This has two types -:

a) Folding shift method -

- The input key is divided into many parts and each part has the same number of digits.
- The last part may have lesser digits than the other parts.
- The individual parts are then added to generate a hash key and if the sum has a carry generated, then that is discarded in the hash value.
- The technique ~~of~~ essentially 'folds' the input key to generate the hash value.

Q.No.						TOTAL
E						
M						

प्र. क्र./Q.No.

Example:

- $k = 12345$
- After folding, $k = 12, 34, 5$
- Adding the individual parts.
  $s = 12 + 34 + 5$
  $= 51$
- $H(k) = 51$
- $k = k_1, k_2, \ldots k_n$
  $s = k_1 + k_2 + \ldots + k_n$
  $H(k) = s$

b) Folding boundary:

- This is same as folding shift but the 1st & last parts are reversed.

- Example:

$k = 12345 = 12, 34, 5$

$\Rightarrow k = 21, 34, 5$

$\Rightarrow s = 60$

$\Rightarrow H(k) = 60$

iii) Multiplication method.

- $H(k) = \text{floor}[M * (kA \bmod 1)]$

~~mod~~ size of hash table

k = input key
A = a pre-defined constant (default A = 0.6180)
floor() function gives the largest integer less than or equal to the product.

- Example:

k = 107, M = 50, A = 0.6180 (default)

$\Rightarrow H(107) = \text{floor}[50 * (107 \times 0.6180) \% 1]$

$= \text{floor}[50 * (66.12 \% 1)]$

$= \text{floor}[50 * 0.12]$

$= \text{floor}[6]$

$= 6$

Q.No.						TOTAL
E						
M						

Savitribai Phule Pune University

प्र. क्र./Q.No.

Q8 - (b)

- Input = [4371, 1323, 6173, 4199, 4344, 9679, 1989]

M = 10 (size of table)

$H(k) = k \% 10$

i) Linear Probing :

- $H(4371) = 4371 \% 10$
  $= 1$
- $H(1323) = 1323 \% 10$
  $= 3$
- $H(6173) = 6173 \% 10$
  $= 3$ (collision)

0	
1	4371
2	
3	1323 → collision
4	
5	
6	
7	
8	
9	

(1)

⇒ Linear Probing searches for the next empty slot from index = 3 onwards and places 6173 in index = 4

⇒ $H(6173) = 4$

- $H(4199) = 4199 \% 10$
  $= 9$
- $H(4344) = 4344 \% 10$
  $= 4$ (collision

0	
1	4371
2	
3	1323
4	6173, 4344 → collision
5	
6	
7	
8	
9	4199

(2)

प्र. क्र./Q.No.

Next empty slot = 5

- $H(9679) = 9679 \% 10$
  $= 9$ (collision)

⇒ Next empty ~~slo~~ slot = 0

⇒ $H(9679) = 0$

0	
1	4371
2	
3	1323
4	6173
5	4344
6	
7	
8	
9	4199 → collision

3

- $H(1989) = 1989 \% 10$
  $= 9$

⇒ Next empty slot = 2

∴ Final Hash Table:

0	9679 ~~1989~~
1	4371
2	
3	1323
4	6173
5	4344
6	
7	
8	
9	4199, 1989 → collision

4

0	9679
1	4371
2	1989
3	1323
4	6173
5	4344
6	
7	
8	
9	4199

6

5

Q. No./Q.No.

ii. ~~Qua~~ Quadratic Probing :

$\cancel{H'(k) =}$ $H'(k,i) = (H(k) + i^2) \% M$

$H(k) = k \% M$
$i = 1$ to $M-1$ (only for collision)

- $H(4371) = 4371 \% 10$
  $= 1$

- $H(1323) = 1323 \% 10$
  $= 3$

- $H(6173) = 6173 \% 10$
  $= 3$ (collision)

0	
1	4371
2	
3	1323, ~~7~~ 6173 → collision
4	→ ~~next empty slot~~
5	
6	
7	
8	
9	

$\Rightarrow H'(6173) = [H(6173) + 1] \% 10$
$= (3+1) \% 10$
$= 4$

0	
1	4371
2	
3	1323
4	6173, 4344 → collision
5	→ ~~next empty~~ slot
6	
7	
8	
9	4199

- $H(4199) = 4199 \% 10$
  $= 9$

- $H(4344) = 4344 \% 10$ $\Rightarrow H'(4344) = [H(4344)+1] \% 10$
  $= 4$ (collision) $= 5 \% 10 = 5$

0	
1	4371
2	
3	1323
4	~~4344~~ 6173
5	4344
6	
7	
8	
9	4199, 9679 → collision

- $H(9679) = 9679 \% 10 = 9$ (collision) ~~$\Rightarrow H'(9679) =$~~

~~$\Rightarrow H'(9679, 2) = [H(9679)+4] \% 10$~~

प्र. क्र./Q.No.

$H'(9679,1) = [H(9679)+1] \% 10$
$= 0$

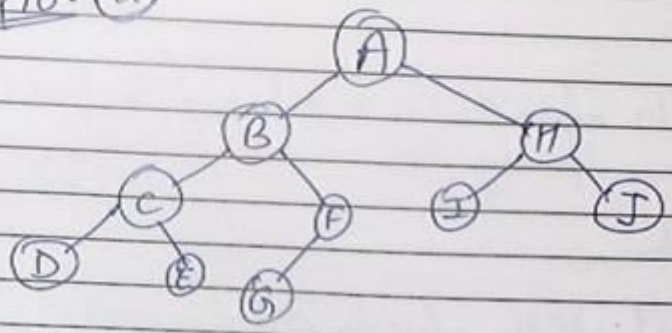
0	9679
1	4371
2	
3	1323
4	6173
5	4344
6	
7	
8	
9	4199

$H'(1989,2) = [H(1989)+4] \% 10$
$= (9+4) \% 10$
$= 3$ (collision)

$H'(1989,3) = (9+9) \% 10$
$= 8$

0	9679
1	4371
2	
3	1323
4	6173
5	4344
6	
7	
8	1989
9	4199

Q10 - (a)



i) Inorder (Left - Root - Right) :

DCEBGFAIHJ

ii) Pre-Order (Root - Left - Right) :

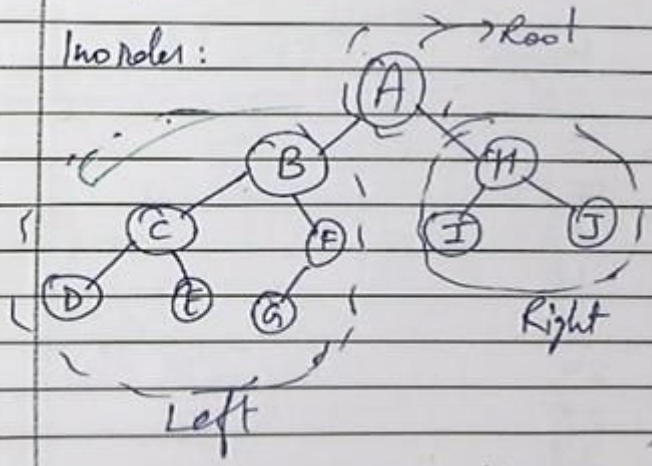
6

ABCDEFGHIJ

iii) Post-Order (~~B~~ Left - Right - Root)

~~DEBGFB~~ DECGFBIJHA

Inorder :



प्र. क्र./Q.No. (b)

- Kruskal's algorithm is a technique to find the Minimum Spanning Tree.

- Algorithm.

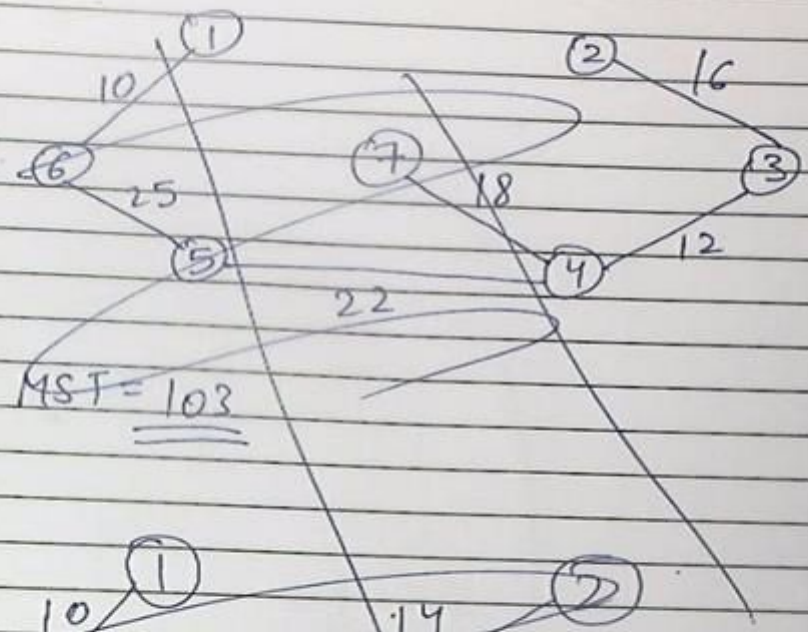
i. ~~Arr~~ Sort all the edges in ascending order.
ii. ~~For a~~ Create a disjoint set for each vertex.
iii. For an edge (u, v) in sorted edge list:
- If u and v are in different sets, add the edge (u, v) to MST.
- Find union of (u, v)

iv) Find the next minimum weight edge which does not form a cycle.

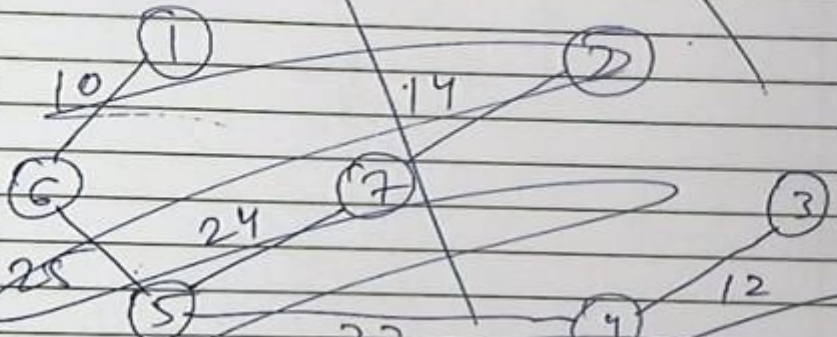
v.) Repeat iii & iv until MST has V-1 edges.

MST:

	Weight
~~1-2~~ 1-6	10
3-4	12
2-~~3~~7	~~15~~ 14
2-3	16
4-7	18
4-5	22
5-7	24
5-6	25
1-2	28

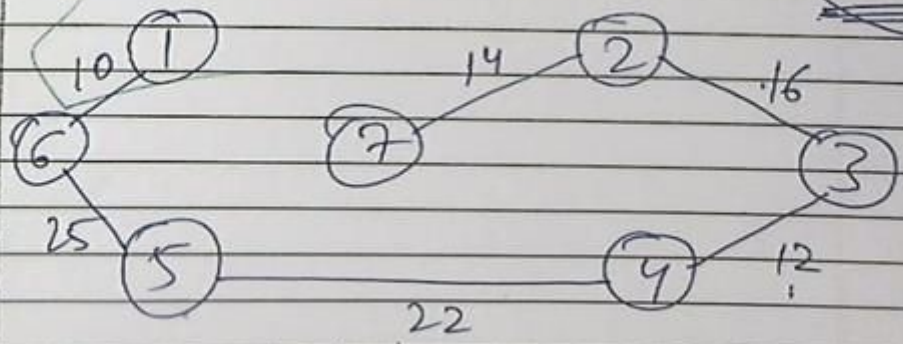


MST = 103



MST = 99

MST = 99



6

Final Answer

SPPU- 32/ 36

	Q.No.					
E						
M						

Savitribai Phule Pune University

प्र. क्र./Q.No.

प्र. /Q.No.

SPPU-34/36

	Q.No.					
E						
M						

Savitribai Phule Pune University

प्र. क्र./Q.No.

प्र./Q.No.

SPPU-36/36

	Q.No.						TOTAL
E							
M							

Savitribai Phule Pune University

Q.No.